

| 유저 적응형 AI 전투 시스템

강화학습 협동 NPC

PPO 기반 5vs5 멀티에이전트 강화학습 시스템. 2단계 학습 파이프라인으로 다양한 플레이어 스타일에 적응하는 협동 NPC 구현. Python-Unity 실시간 연동.



2인 개발

AI 학습, Unity 연동



Unity 2022

3D 시각화



5+ Systems

Advanced Features



2개월

제작 기간



프로젝트 보기



GitHub



논문 PDF

PROJECT OVERVIEW

게임 개요

PPO 기반 5vs5 멀티에이전트 강화학습 시스템. 2단계 학습 파이프라인으로 다양한 플레이어 스타일에 적응하는 협동 NPC 구현. Python-Unity 실시간 연동.



핵심 게임플레이

- ✓ PPO 기반 5vs5 멀티에이전트 강화학습
- ✓ 2단계 학습 파이프라인 (Self-Play → 협동 NPC)
- ✓ 3가지 플레이어 정책으로 다양한 스타일 대응
- ✓ UDP/TCP 실시간 Python-Unity 연동



개발 환경

- ✓ 엔진: Unity 6
- ✓ 기간: 2024.11
- ✓ 플랫폼: Windows PC



개발 성과

- ✓ 협동 NPC 학습으로 평균 탱커 거리 71% 감소 (10.0 → 2.91)
- ✓ UDP/TCP 분리 통신으로 약 1ms 지연 실시간 연동
- ✓ 3가지 플레이어 정책으로 다양한 스타일 대응

TECHNOLOGY STACK

기술 스택

PPO 기반 5vs5 멀티에이전트 강화학습 시스템

ML Framework

PT

PyTorch 2.0
신경망 학습

PPO

PPO + GAE
정책 최적화

게임 엔진

U

Unity 2022.3
3D 시각화

C#

C#
클라이언트 로직

네트워크

UDP

UDP Socket
게임 상태 스트리밍

TCP

TCP Socket
플레이어 입력 전송

AI 시스템

AC

Actor-Critic

229→256→256→12

MA

Multi-Agent

5vs5 전투

CORE FEATURES

핵심 기능 구현

6가지 주요 시스템과 기술적 구현 상세

```

class GoalTankPolicy:
    """탐험형 플레이어: 목표 지점으로 이동"""
    def __init__(self, env):
        self.env = env
        self.goal = None

    def get_action(self, obs, unit):
        if self.goal is None or self._reached_goal(unit):
            self.goal = self._random_goal()
        return self._move_toward_goal(unit)

class ModelTankPolicy:
    """숙련 플레이어: 학습된 모델 행동"""
    def __init__(self, model_path):
        self.model = torch.load(model_path)

    def get_action(self, obs, unit):
        with torch.no_grad():
            return self.model(obs).argmax().item()

class ConditionalTankPolicy:
    """반응형 플레이어: 상황 기반 행동"""
    def get_action(self, obs, unit):
        nearby_enemies = self._get_nearby_enemies(unit)
        if nearby_enemies:
            return self._engage_enemy(unit, nearby_enemies[0])
        return self._patrol(unit)

```

1. 탱커 정책 시스템

→ 다양한 플레이어 스타일 대응

- 고정된 플레이어 모델로는 특정 스타일에만 최적화되는 문제 발생
- 3가지 탱커 정책 랜덤 선택 시스템 설계

→ 정책 구성

- GoalTankPolicy (50%): 목표 지점 이동 (탐험형)
- ModelTankPolicy (15%): 1단계 학습 행동 (숙련자)
- ConditionalTankPolicy (35%): 상황 대응 (반응형)

```
def calculate_coop_reward(npc_unit, tank_unit, action_result):
    """협동 보상 계산"""
    reward = 0

    # 기본 전투 보상
    if action_result.killed_enemy:
        reward += 50
    if action_result.damaged_enemy:
        reward += 10

    # 탱커 거리 기반 협동 보상
    distance = manhattan_distance(npc_unit.pos, tank_unit.pos)

    if distance <= 0:
        reward += 15 # 매우 근접
    elif distance <= 2:
        reward += 10 # 근접
    elif distance <= 4:
        reward += 5 # 적정 거리
    else:
        reward -= 5 # 너무 멀리 떨어짐

    # 협공 보상 (탱커 근처에서 공격 시)
    if action_result.attacked and distance <= 3:
        reward += 20

    return reward
```

2. 협동 보상 설계

→ 문제

- 승패 보상만으로는 NPC가 플레이어를 따라다니지 않음

→ 해결: 거리 비례 협동 보상

- 0칸: +15 (매우 근접)
- 2칸: +10
- 4칸: +5
- 5칸+: -5 (패널티)

→ 결과

- 평균 탱커 거리: 10.0 → 2.91 감소
- 협동 행동 학습 검증 완료

```
public class UdpReceiver : MonoBehaviour
{
    private UdpClient udpClient;
    private Thread receiveThread;
    private GameState latestState;

    void Start()
    {
        udpClient = new UdpClient(5005);
        receiveThread = new Thread(ReceiveData);
        receiveThread.IsBackground = true;
        receiveThread.Start();
    }

    void ReceiveData()
    {
        IPEndPoint remoteEP = new IPEndPoint(IPAddress.Any, 0);

        while (true)
        {
            byte[] data = udpClient.Receive(ref remoteEP);
            string json = Encoding.UTF8.GetString(data);
            latestState = JsonUtility.FromJson<GameState>(json);
        }
    }

    void Update()
    {
        if (latestState != null)
        {
            gameViewer.UpdateView(latestState);
        }
    }
}
```

3. Python-Unity 연동

→ 실시간 통신 아키텍처

- 게임 상태 (Python→Unity): UDP - 속도 우선, 프레임 손실 허용
- 플레이어 입력 (Unity→Python): TCP - 신뢰성 우선, 입력 손실 불가

→ 성능

- 약 1ms 지연의 실시간 연동 구현
- UDP 5005 포트: 상태 스트리밍
- TCP 5006 포트: 입력 수신

```
public class PlayerInputSender : MonoBehaviour
{
    private TcpClient tcpClient;
    private NetworkStream stream;

    void Start()
    {
        tcpClient = new TcpClient("127.0.0.1", 5006);
        stream = tcpClient.GetStream();
    }

    void Update()
    {
        int action = GetPlayerAction();
        if (action >= 0)
        {
            SendAction(action);
        }
    }

    int GetPlayerAction()
    {
        // 이동: WASD
        if (Input.GetKeyDown(KeyCode.W)) return 0;
        if (Input.GetKeyDown(KeyCode.S)) return 1;
        if (Input.GetKeyDown(KeyCode.A)) return 2;
        if (Input.GetKeyDown(KeyCode.D)) return 3;

        // 공격: 방향키 + 대각선
        if (Input.GetKeyDown(KeyCode.UpArrow)) return 4;
        if (Input.GetKeyDown(KeyCode.DownArrow)) return 5;
        // ... 8방향 공격

        return -1;
    }

    void SendAction(int action)
    {
        string json = $"{{\"action\": {action}}}\";
        byte[] data = Encoding.UTF8.GetBytes(json);
        stream.Write(data, 0, data.Length);
    }
}
```

4. 플레이어 입력 전송

→ TCP 기반 입력 처리

- 신뢰성 있는 입력 전송 보장
- 플레이어 탱커 유닛 제어
- 실시간 행동 선택 및 전송

→ 입력 형식

- action: 0-11 (이동 4방향 + 공격 8방향)
- unit_id: 제어 대상 유닛

```
class ActorCritic(nn.Module):
    def __init__(self, obs_dim=229, act_dim=12, hidden=256):
        super().__init__()

        # Shared feature extractor
        self.shared = nn.Sequential(
            nn.Linear(obs_dim, hidden),
            nn.ReLU(),
            nn.Linear(hidden, hidden),
            nn.ReLU()
        )

        # Actor head (policy)
        self.actor = nn.Linear(hidden, act_dim)

        # Critic head (value)
        self.critic = nn.Linear(hidden, 1)

    def forward(self, obs):
        features = self.shared(obs)
        return self.actor(features), self.critic(features)

    def get_action(self, obs):
        logits, value = self.forward(obs)
        probs = F.softmax(logits, dim=-1)
        dist = Categorical(probs)
        action = dist.sample()
        return action, dist.log_prob(action), value
```

5. PPO 에이전트

→ Actor-Critic 네트워크

- 입력: 229차원 (유닛 상태, 주변 정보)
- 히든: 256 → 256
- 출력: 12개 행동 (이동 4 + 공격 8)

→ 학습 설정

- Learning Rate: 3e-4
- Gamma: 0.99
- GAE Lambda: 0.95
- Clip Range: 0.2

```
# 학습 결과 요약
training_results = {
    'stage1_selfplay': {
        'total_episodes': 12000,
        'team_a_winrate': 0.502,
        'team_b_winrate': 0.494,
        'training_time_hours': 6.1,
        'avg_fps': 905
    },
    'stage2_coop': {
        'total_episodes': 12750,
        'reward_change': (-1297, 1739),
        'avg_tank_distance': (10.0, 2.91),
        'winrate_balance': (0.47, 0.52)
    }
}

# 협동 행동 학습 검증
# 평균 탱커 거리 71% 감소 (10.0 → 2.91)
# NPC가 플레이어 탱커를 따라다니며 협동 전투 수행
```

6. 학습 결과

→ 1단계: Self-Play

- 총 에피소드: 12,000
- Team A 승률: 50.2%
- Team B 승률: 49.4%
- 학습 시간: 6.1시간

→ 2단계: 협동 학습

- 총 에피소드: 12,750
- 평균 보상: -1,297 → +1,739
- 평균 탱커 거리: 10.0 → 2.91
- 승률 균형: 47:52

디자인 패턴

확장 가능하고 유지보수하기 쉬운 코드 아키텍처



Actor-Critic

PPO 기반 정책 최적화와 가치 함수 학습

✓ 적용 완료



Strategy

GoalTankPolicy, ModelTankPolicy, ConditionalTankPolicy 전략 교체

✓ 적용 완료



Observer

UDP 상태 스트리밍, TCP 입력 수신 비동기 처리

✓ 적용 완료



Singleton

GameManager, NetworkManager 전역 관리

✓ 적용 완료

ACHIEVEMENTS

주요 성과

프로젝트를 통해 달성한 기술적 성과와 학습 내용



기술적 성과

- ★ 협동 NPC 학습으로 평균 탱커 거리 71% 감소 (10.0 → 2.91)
- ★ UDP/TCP 분리 통신으로 약 1ms 지연 실시간 연동
- ★ 3가지 플레이어 정책으로 다양한 스타일 대응



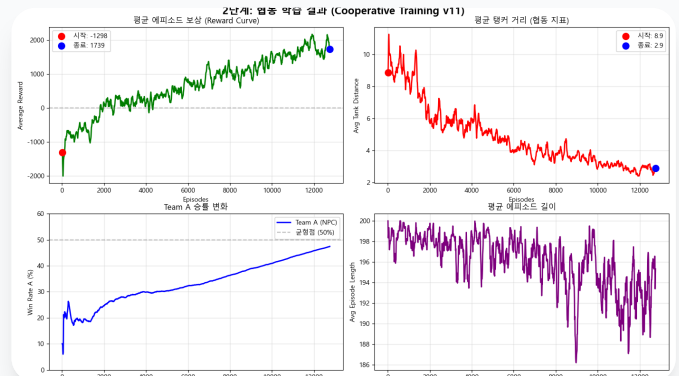
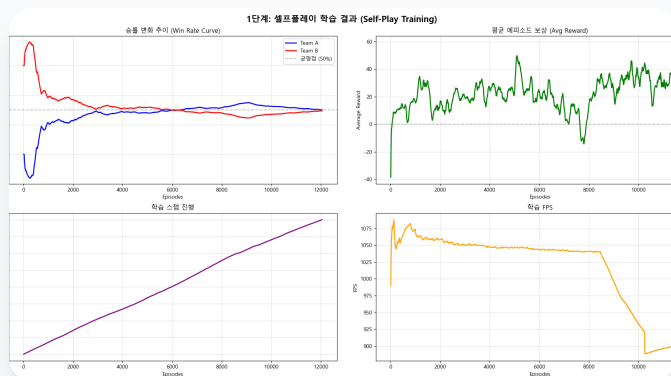
배운점

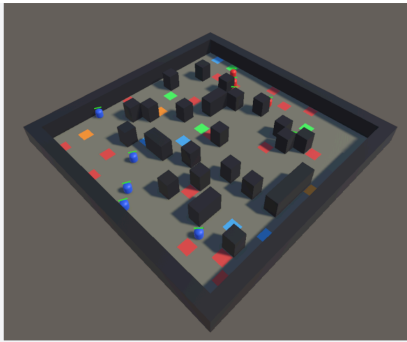
- ★ 강화학습에서 보상 설계의 중요성 체감 (협동 보상)
- ★ UDP/TCP 프로토콜 선택 기준 학습 (속도 vs 신뢰성)
- ★ 멀티에이전트 환경에서 Self-Play의 효과 검증
- ★ Python-Unity 연동 아키텍처 설계 경험

SCREENSHOTS & VIDEOS

게임 미디어

게임 플레이 스크린샷과 영상





실제 테스트한 유니티 3D 데모 환경

🎮 강화학습 협동 NPC

로 개발한 유저 적응형 AI 전투 시스템 포트폴리오

© All Rights Reserved

프로젝트 링크

 [GitHub Repository](#)

 [논문 PDF](#)

 [홈으로 돌아가기](#)

기술 스택

Python

PyTorch

Unity

C#

Reinforcement Learning